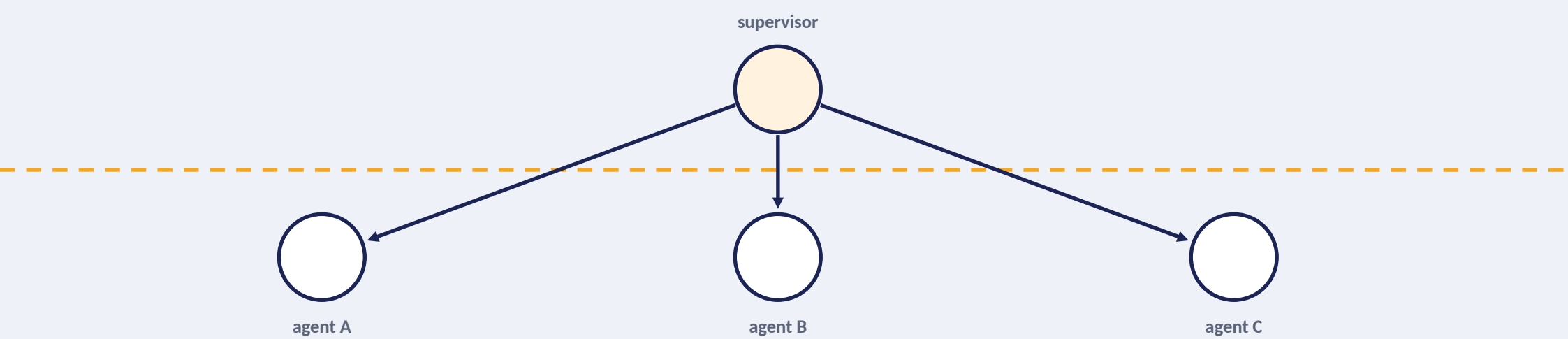
LANGGRAPH · CHAPTER 8 · FLASHCARDS

# Acting Agents and Multi-Agent Handoffs

*Code guards, agents hand off*

12 code cards on guarded tools, handoffs, subgraphs and a supervisor



# A guard that lives in the tool

*The docstring tells the model the rule; the if statement in the tool enforces it*

- send_email must take a to, so it checks it against an allow-list that defaults to your address.
- A refusal comes back as text the model can explain, not an exception.

PYTHON

```python
def _allowed_recipients() -> set[str]:
    configured = (os.getenv("ALLOWED_EMAIL_RECIPIENTS")
                  or os.getenv("GMAIL_ADDRESS", ""))
    return {a.strip().lower() for a in configured.split(",") if a.strip()}


@tool
def send_email(to: str, subject: str, body: str) -> str:
    """Send an email from the user's Gmail account. `to` must be approved."""
    if to.strip().lower() not in _allowed_recipients():
        return f"Refused: {to} is not an approved recipient."   # code decides
    if os.getenv("NOTIFY_DRY_RUN") == "1":
        print(f"[DRY RUN] Email -> to={to!r} subject={subject!r}\n{body}")
        return "Dry run: the email was not really sent."
    ...   # read GMAIL_ADDRESS / GMAIL_APP_PASSWORD, send with smtplib.SMTP_SSL


send_email.invoke({"to": "stranger@example.com", "subject": "Hi", "body": "..."})
# 'Refused: stranger@example.com is not an approved recipient.'
```

# The model writes, code addresses

*Telegram and Discord tools take only text; the chat and channel come from config*

- Tokens and webhook URLs are read inside the tool, so they never appear in tool_calls or traces.
- NOTIFY_DRY_RUN=1 prints what would be sent: run every new acting agent this way first.

PYTHON

```python
@tool
def send_telegram_message(text: str) -> str:
    """Send a short text message to the user's own Telegram chat.
    It always goes to the user; you cannot choose another recipient."""
    if os.getenv("NOTIFY_DRY_RUN") == "1":
        print(f"[DRY RUN] Telegram -> {text!r}")
        return "Dry run: the Telegram message was not really sent."
    token = os.getenv("TELEGRAM_BOT_TOKEN")          # secrets come from .env
    chat_id = os.getenv("TELEGRAM_CHAT_ID")          # the destination too
    if not token or not chat_id:
        return "Telegram is not configured (... missing)."
    response = requests.post(f"https://api.telegram.org/bot{token}/sendMessage",
                             json={"chat_id": chat_id, "text": text}, timeout=10)
    return "Telegram message sent." if response.ok else "Telegram error ..."

print(send_telegram_message.args)     # the whole contract the model sees
# {'text': {'title': 'Text', 'type': 'string'}}    <- no token, no chat id

# send_discord_message(text) has the same shape: webhook URL from .env
```

# A limit on the agent, in code

*"Send each message once" in the prompt is a request; a tool-call limit is a rule*

- In the real dry run the prompt said once, yet Telegram and Discord each fired twice.
- With no tool_name the limit counts all tools together: a second send is refused.

PYTHON

```python
from tools import NOTIFY_TOOLS     # the three send tools, one file each

agent = create_agent(
    model="openai:gpt-4o-mini",
    tools=NOTIFY_TOOLS,
    system_prompt=SYSTEM_PROMPT,     # "...Send each message once..."
    # At most one email per run, whatever the model decides.
    middleware=[ToolCallLimitMiddleware(tool_name="send_email",
                                        run_limit=1)],
)
result = agent.invoke({"messages": [HumanMessage(request)]})

# To make "once" a rule on every channel:
middleware=[ToolCallLimitMiddleware(run_limit=1)]    # any tool, one call per run
# Checked offline: of two parallel send_telegram_message calls the first
# runs; the second gets "Tool call limit exceeded..." and sends nothing.
```

# A handoff in plain Python

*Run the analyst, copy its last message into a string, paste it into the writer's prompt*

- The writer gets only the conclusion, not the analyst's tool calls: usually what you want.
- Between the agents there is no graph: nothing to draw, stream, checkpoint or trace.

PYTHON

```python
analyst = create_agent(
    model="openai:gpt-4o-mini",
    tools=[multiply, subtract],
    system_prompt=("You are a financial analyst. Calculate revenue and profit. "
                   "Always use the provided tools for arithmetic."),
)
writer = create_agent(
    model="openai:gpt-4o-mini",
    tools=[word_count],
    system_prompt=("You are a report writer. Turn the supplied analysis into a "
                   "clear report of at most 80 words. "
                   "Use word_count to verify the length."),
)

result1 = analyst.invoke({"messages": [HumanMessage(REQUEST)]})
analysis = last_text(result1["messages"])        # the handoff: just a string

result2 = writer.invoke({"messages": [HumanMessage(
    "The first agent produced this analysis:\n\n"
    f"{analysis}\n\nWrite the final report.")]})
print(last_text(result2["messages"]))
```

# A handoff through graph state

*Each agent runs inside a node; the handoff becomes an edge and the result a state field*

- The writer also sees the original request, because request is still in state.
- invoke returns request, analysis and report: the intermediate result is kept.

PYTHON

```python
class State(TypedDict):
    request: str
    analysis: str       # written by the analyst node
    report: str         # written by the writer node
def analyst_node(state: State) -> dict:
    result = analyst.invoke({"messages": [HumanMessage(state["request"])]})
    return {"analysis": result["messages"][-1].content}       # a partial update
def writer_node(state: State) -> dict:
    prompt = (f"Original request:\n{state['request']}\n\n"
              f"Analysis from the analyst:\n{state['analysis']}\n\n"
              "Write the final report.")
    result = writer.invoke({"messages": [HumanMessage(prompt)]})
    return {"report": result["messages"][-1].content}

builder = StateGraph(State)
builder.add_node("analyst", analyst_node)
builder.add_node("writer", writer_node)
builder.add_edge(START, "analyst")
builder.add_edge("analyst", "writer")      # the handoff is now an edge
builder.add_edge("writer", END)
graph = builder.compile()
result = graph.invoke({"request": REQUEST, "analysis": "", "report": ""})
```

# Two agents in parallel, one combiner

*Fan the same question out to two agents, then merge their reports in one node*

- The node function is the boundary: question in, last message out, tool calls stay inside.
- combine has two incoming edges, so it runs once, after the slower agent finishes.

PYTHON

```python
class TripState(TypedDict):
    question: str          # set by the caller; both agents read it
    weather_report: str    # written only by weather_node
    budget_report: str     # written only by budget_node
    final_answer: str      # written only by combine_node

def weather_node(state: TripState) -> dict:
    result = weather_agent.invoke(
        {"messages": [HumanMessage(state["question"])]})       # parent -> subgraph
    return {"weather_report": result["messages"][-1].content}  # subgraph -> parent

# budget_node is the same shape and writes budget_report

builder = StateGraph(TripState)
builder.add_node("weather", weather_node)
builder.add_node("budget", budget_node)
builder.add_node("combine", combine_node)     # an LLM with no tools: it only reads
for agent_name in ("weather", "budget"):
    builder.add_edge(START, agent_name)       # fan-out: same question to both
    builder.add_edge(agent_name, "combine")   # fan-in: combine waits for both
builder.add_edge("combine", END)
graph = builder.compile()
```

# A compiled graph as the node

*Pass a compiled graph to add_node and the keys both states declare flow in and out*

- No translation lines: question flows in and weather_report flows out by name.
- scratch exists only in the subgraph, so the parent never receives it.

PYTHON

```python
class WeatherState(TypedDict):
    question: str                          # shared with the parent (read)
    weather_report: str                    # shared with the parent (written)
    scratch: Annotated[list[str], add]     # private: the parent has no such key
weather_subgraph = weather_builder.compile()   # START -> look_up -> write_up -> END

class TripState(TypedDict):
    question: str
    weather_report: str
parent_builder = StateGraph(TripState)
parent_builder.add_node("weather", weather_subgraph)   # the compiled graph IS it
parent_builder.add_edge(START, "weather")
parent_builder.add_edge("weather", END)
parent_graph = parent_builder.compile()

question = {"question": "Montreal this weekend?"}
print(parent_graph.invoke(question))
# {'question': 'Montreal this weekend?',
#  'weather_report': 'Report for: Montreal this weekend?'}
print(weather_subgraph.invoke(question)["scratch"])
# ['looked up the forecast', 'wrote the report']
```

PITFALL · PARALLEL COLLISIONS

# Two writers, one box

*Two parallel nodes writing the same plain key in one step stop the run with an error*

- A subgraph passed to add_node hands back every shared key, even ones it only read.
- Fix: one field per agent, or an Annotated[list[str], add] reducer on the shared key.

PYTHON

```python
# 1. Two shared-key subgraphs in parallel: both hand back question
parent.add_node("weather", weather_subgraph)    # shares question, weather_report
parent.add_node("budget", budget_subgraph)      # shares question, budget_report
parent.add_edge(START, "weather"); parent.add_edge(START, "budget")
# InvalidUpdateError: At key 'question': Can receive only one value per step.

# 2. Exercise 8.3: both agent nodes write one report field
class TripState(TypedDict):
    question: str
    report: str                                  # weather AND budget write it
# InvalidUpdateError: At key 'report': Can receive only one value per step.
#   Use an Annotated key to handle multiple values.

# Fix A: weather_report and budget_report, one writer each (Listing 8.7)
# Fix B: report: Annotated[list[str], add], each node returns a one-item list

# Both errors reproduced on langgraph 1.2.6 with deterministic nodes
```

# Agents wrapped as tools

*Wrap each specialist in a tool and a main agent calls only the ones a question needs*

- Same two-line boundary, inside a tool: only the subagent's answer reaches the main agent.
- The model now picks who runs, so you lose the guarantee that both specialists run.

PYTHON

```python
@tool
def ask_weather_agent(question: str) -> str:
    """Ask the weather specialist about the weather for a trip."""
    result = weather_agent.invoke({"messages": [HumanMessage(question)]})
    return result["messages"][-1].content       # only the answer goes back

# ask_budget_agent wraps budget_agent the same way

main_agent = create_agent(model="openai:gpt-4o-mini",
                          tools=[ask_weather_agent, ask_budget_agent])

main_agent.invoke({"messages": [HumanMessage(
    "How much is one hotel night in Ottawa?")]})
# expected: only ask_budget_agent is called; the subagent's own tool calls
# stay in its messages, not in the main agent's (Exercise 8.4)
```

# A structured decision, capped in code

*The supervisor returns a Literal choice as a Command, and a counter guarantees it stops*

- next is a Literal, so the reply can only be nutrition, cost or finish: no misspelled routes.
- feedback is overwritten on every hop, so one specialist never gets the other's notes.

PYTHON

```python
MAX_WORKER_RUNS = 4          # two specialists, one retry each, then we stop
class Decision(BaseModel):
    next: Literal["nutrition", "cost", "finish"]
    feedback: str = ""       # what to fix when sending a specialist back
supervisor_llm = (init_chat_model(MODEL, temperature=0)
                  .with_structured_output(Decision))
def supervisor_node(state: KitchenState
                    ) -> Command[Literal["nutrition", "cost", "writer"]]:
    worker_runs = state.get("worker_runs", 0)
    if worker_runs >= MAX_WORKER_RUNS:          # code, not the prompt, ends the loop
        return Command(goto="writer",
                       update={"path": ["writer (run limit reached)"]})
    decision = supervisor_llm.invoke(SUPERVISOR_PROMPT.format(
        request=state["request"],
        nutrition_report=state.get("nutrition_report", "(none yet)"),
        cost_report=state.get("cost_report", "(none yet)")))
    if decision.next == "finish":
        return Command(goto="writer", update={"path": ["writer"]})
    return Command(goto=decision.next,
                   update={"feedback": decision.feedback,
                           "worker_runs": worker_runs + 1, "path": [decision.next]})
```

# Every specialist reports back

*Specialist nodes always return to the supervisor, which alone picks the next hop*

- On a retry the task carries the specialist's previous report and the supervisor's feedback.
- Supervisor has no outgoing edge: its Command picks nutrition, cost or writer.

PYTHON

```python
def run_specialist(agent, state: KitchenState, report_field: str) -> dict:
    task = f"Request:\n{state['request']}"
    previous_report = state.get(report_field)
    if previous_report and state.get("feedback"):        # a retry
        task += (f"\n\nYour previous report:\n{previous_report}"
                 f"\n\nThe supervisor asked you to fix this:\n{state['feedback']}")
    result = agent.invoke({"messages": [HumanMessage(task)]})
    return {report_field: result["messages"][-1].content}   # only the conclusion
def nutrition_node(state: KitchenState) -> dict:
    return run_specialist(nutrition_agent, state, "nutrition_report")
# cost_node: the same, with cost_agent and "cost_report"
builder = StateGraph(KitchenState)
builder.add_node("supervisor", supervisor_node)
builder.add_node("nutrition", nutrition_node)
builder.add_node("cost", cost_node)
builder.add_node("writer", writer_node)
builder.add_edge(START, "supervisor")
builder.add_edge("nutrition", "supervisor")    # every specialist reports back
builder.add_edge("cost", "supervisor")
builder.add_edge("writer", END)
graph = builder.compile()
```

PITFALLS

# A cut spoke and the run cap

*A missing return edge stops the run silently; a supervisor without a cap never stops*

- A node with no outgoing edge still compiles: the run just ends after it, with no error.
- The prompt asks the supervisor to finish; only MAX_WORKER_RUNS guarantees it does.

PYTHON

```python
# 1. Exercise 8.5: delete one spoke
# builder.add_edge("cost", "supervisor")       <- removed
graph = builder.compile()                      # no complaint
result = graph.invoke({"request": REQUEST})
# path: ['nutrition', 'cost'] and then nothing:
# no writer, no recipe card, no final_answer in the result

# 2. A supervisor that is never satisfied, with the cap in place
# route: nutrition -> nutrition -> nutrition -> nutrition
#        -> writer (run limit reached)            worker_runs = 4

# Both checked offline on langgraph 1.2.6 with a scripted supervisor
```